



CUDA

Date 01/10/2026

Global ID:

Now, we can find:

- * $\text{threadIdx.x} \times \text{blockDim.x} \rightarrow \text{thread No. for that Block}$
- * $\text{blockIdx.x} \times \text{blockDim.x} \rightarrow \text{Block No.}$
- * $\text{blockDim.x} \rightarrow \text{Total threads.}$

But, we don't have a Unique id for each thread.

Ex) Block 0 : thread 0 $\rightarrow$ T0
 thread 1 $\rightarrow$ T1
 :
 :
 Block 1 : thread 0 $\rightarrow$ T0
 thread 1 $\rightarrow$ T1

They are of same values

Therefore, to provide each thread with a Unique ID, we use

Global ID

Ex) Block 0:

Thread 0 $\rightarrow$ 0

Thread 1 $\rightarrow$ 1

Thread 2 $\rightarrow$ 2

Thread 3 $\rightarrow$ 3

Block 1:

T0 $\rightarrow$ 4

T1 $\rightarrow$ 5

T2 $\rightarrow$ 6

T3 $\rightarrow$ 7

Block 2:

T0 $\rightarrow$ 8

T1 $\rightarrow$ 9

T2 $\rightarrow$ 10

T3 $\rightarrow$ 11

You must learn to accept defeat without being defeated.

Now, in the previous case, the ThreadDim is 4. At each Block the start was a multiple of 4, i.e., ThreadDim. With this we can create a formula:

$$\text{int globalid} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

Ex) $\text{global id} = \text{Block } 1 * 4 \text{ threads} + 2$
 $= 1 * 4 + 2 = 6$

Therefore, the ^{Global id of} ~~thread~~ 1st block, 2nd thread will be 6 when number of threads is 4.

How it connects to image processing:

P0	P1	P2	P3
P4	P5	P6	P7

Row, thread
 0 → process P0
 T1 → P1
 T2 → P2...

Dikewise

Each thread will process a pixel of the image.

Vector Addition:

$$A = [10, 20, 30, 40] \Rightarrow A + B = [11, 22, 33, 45]$$

$$B = [1, 2, 3, 4]$$

Now, while CPU computes this, it will do sequential processing

$$C[0] = A[0] + B[0]$$

$$C[1] = A[1] + B[1]$$

Whereas while GPU processes it, Each thread parallelly splits the work:

$$C[0] = A[0] + B[0] \rightarrow \text{Thread 1}$$

$$C[1] = A[1] + B[1] \rightarrow \text{Thread 2}$$

Global ID tells which thread processes which element.

GPU Memory:

CPU Memory $\rightarrow$ RAM
GPU Memory $\rightarrow$ VRAM / GPU Memory.

Why need?

For that, we need to understand this
Data transfer happens b/w RAM & VRAM

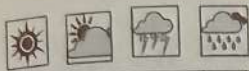
Ex) we create Array:
`int A[4] = {10, 20, 30, 40};`

Now, this A lives in CPU Memory.
When we launch the kernel $\leftarrow \rightarrow$ it needs A to process.

Now A (in CPU Mem) $\rightarrow$ GPU Mem.

Once completed A (in GPU Mem) $\rightarrow$ CPU Mem. (RAM)

This is Host Device Data transfer.



Mo Tu We Th Fr Sa Su

Date / /

Host:

↳ It is CPU. / runs the code in CPU.

↳ Ex)

```
int main() { ...
```

```
... } ⇒ Host code.
```

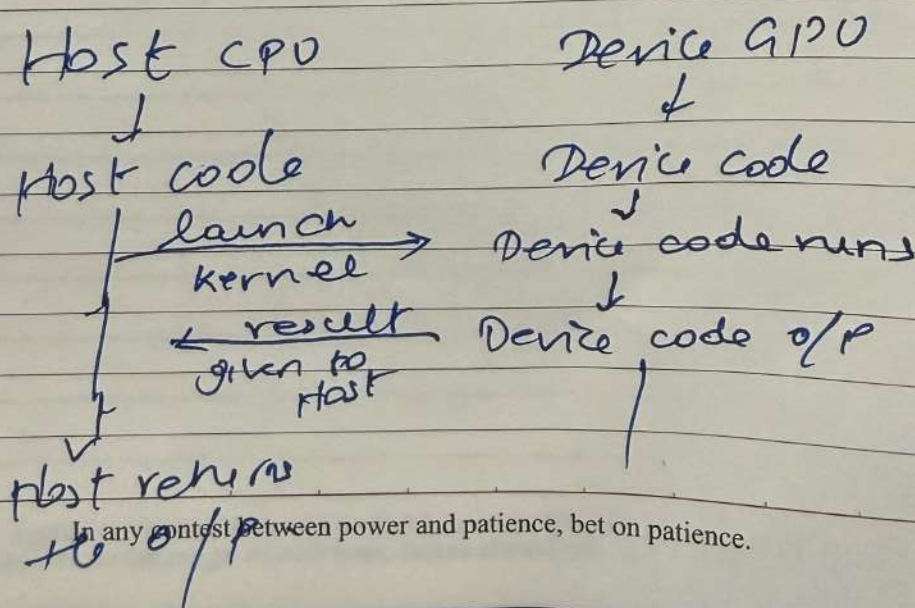
Device:

↳ GPU / code run in GPU

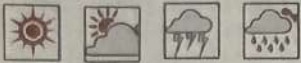
↳ Ex) — global — void kernel().

⇒ Device code that runs on GPU.

~~Now, Host code in CPU starts running, it needs the GPU~~



In any contest between power and patience, bet on patience.



Mo	Tu	We	Th	Fr	Sa	Su
----	----	----	----	----	----	----

Date / /

How GPU get Memory?

→ `CudaMalloc()` → GPU, allocate
Some memory for me.

* `int *d_A;` # `d_A` → Device Array
 `h_A` → host Array

`CudaMalloc(&d_A, 4 * sizeof(int));`